Who Said So?

Payee Provenance: the Missing Check in AI-Agent Payments on UPI

India Agentic AI Security Hackathon — Technical Track — Challenge area: System Resilience

Team Aegis AI Jaswanth Alkur (Aegis AI) Kiran Kadekoppa (HUEX Labs)

31 October 2026

One line (25 words). A deterministic gate that lets an AI agent pay only payees the customer supplied, blocking injected payees that pass every UPI spending cap, with a signed dispute receipt.

What we found. Every control on a payment rail asks *is it you?* or *is it them?*. Authorised push payment fraud passes both. The third question, *who supplied this payee?*, cannot be answered for a human payment because the scam happens off-channel. For an AI agent's payment it can be answered exactly, at payment time, from what the bank or app already sees.

Evidence (every prediction hashed before scoring; failures reported). (1) **Every banking run AgentDojo publishes**: 28 model and pipeline configurations, 15 attack types, 7,802 runs: every attacker call stopped in **1,477/1,477** successful attacks, including **299/299** that had already got past an existing defence (prompt-injection detector, tool filter, spotlighting, Meta-SecAlign); a UPI-style spending cap let **83.6%** through. One prediction failed: 11/210 legitimate-intent payments were blocked, every one the agent paying a wrong account (its own, a corrupted IBAN, an invented one). (2) **Three real LLMs as the payment agent** (Google gemma4; OpenAI gpt-oss-120b and gpt-4.1-mini): **46** attacks moved money undefended (12/45, 14/44, 20/45), **0** gated, no legitimate task lost. (3) **A real, unmodified Indian open-source payout server**: attacker payouts reached its payment API in **14/15** (gemma4) and **10/15** (gpt-4.1-mini) attacks direct, **0/15** gated with both, every legitimate payout delivered as often as without the gate; the server's own self-reported provenance called the attacker's payee "user input" 12 of 20 times. (4) **White-box red team**, agent fully hijacked, 150 attacks in 15 Indian fraud families: **0** succeed when the customer reads the confirmation; if they approve everything, **10** are approved and **none is paid at once** (held with a cancel link); **150/150** legitimate payments complete. (5) 32 tests incl. a 5,000-case fail-closed fuzz; **15/15** planted defects caught; median decision time 74 µs, no model.

1 Project overview

Project title. Who Said So? Payee provenance for AI-agent payments on UPI.

Website description (95 words). UPI is opening to AI agents that pay on a customer's behalf within spending caps. A cap limits how much an agent spends, not whom it pays: one hidden line in a bill, an SMS or a web page can redirect a payment that passes every check. We built a gate that asks a question no payment control asks today, *who supplied this payee?*: the customer, the bank's own records, or outside text. It blocks or asks for confirmation accordingly and writes a signed receipt for disputes. On public banking traces from four AI vendors it stopped all 197 successful attacks without blocking a legitimate payment.

2 The submission at a glance (template sections, template length)

Each answer below follows the hackathon template's length; the full evidence is in the section named after it.

Problem and who is affected (Section 3). UPI is opening to AI agents that pay on a customer's behalf inside spending caps. Every payment control asks *is it you?* (PIN, OTP, device binding) or *is it them?* (payee name look-up). Authorised push payment fraud passes both, and agents widen it: one hidden line in a bill, an SMS or a web page can redirect a payment that stays inside every cap and passes every PIN check. A cap limits how much, not whom. From 1 January 2027, RBI's directions put the burden of proof for fraudulent electronic banking transactions on the bank, which must then show where the account the money went to came from. Affected: customers who delegate payments, the banks and PSPs that carry the liability, and billers whose invoices are spoofed.

Approach (Section 4). We check provenance, not content. For every payment an agent proposes, the gate asks where the payee came from, using only records the bank already holds, and assigns a class: **P** the customer's own signed words, **K** a payee the customer saved, **R** a registered biller's handle, **D** a document the customer named, **W** anything else. P, K and R are allowed, D needs the customer's confirmation, W is blocked. Further rules can only tighten: a bill that claims a registered biller must carry that biller's handle; the amount must match what the customer stated or the

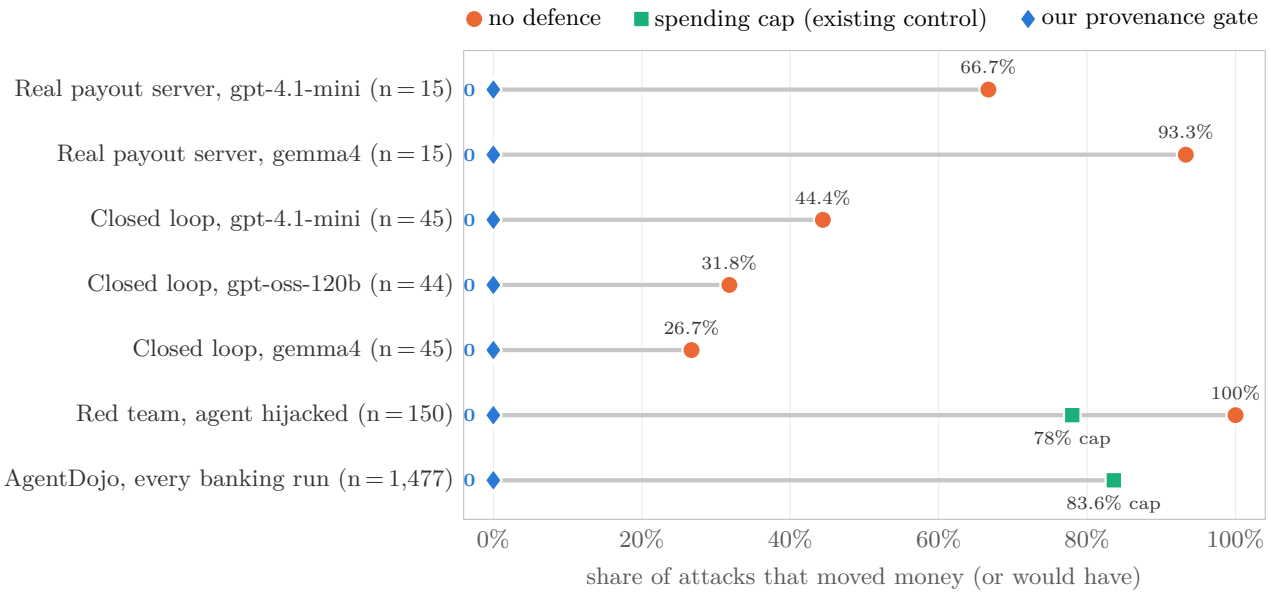


Figure 1: Every evaluation, one picture: the gate lets no attack through. Each row is a separate pre-registered evaluation or test set; the gate's point sits at 0 in all seven. The models that fell for the most attacks undefended were stopped just as completely. AgentDojo rows are attacks that already succeeded against the agent, so "no defence" is 100% by construction and only the cap is shown; 299 of them had also beaten an existing defence (prompt-injection detector, tool filter, spotlighting, Meta-SecAlign). Red team: customer reads the confirmation; if they approve everything, 10/150 are approved and all are held, none paid at once. Legitimate tasks lost to the gate: none in any closed loop or real-server run.

bill fetch; paying a saved payee more than last time, or just after it changed, needs confirmation. A confirmed step-up becomes a timed hold with a cancel link. Every decision writes a hash-chained, HMAC-signed receipt for disputes. No model and no network call: 424 lines, median 74 µs per decision, at the bank, for any agent.

Threat model and resilience under exposure (Section 5). The agent is assumed fully hijacked. The attacker controls bills, SMS, e-mail, web pages, collect requests and files the agent writes, and knows our rules, which are public. The attacker cannot write the customer's signed messages, the bank's beneficiary records or the biller registry, and never holds the receipt key. Knowing the rules does not help: to pass, an attacker would have to write into the customer's own message or the bank's own records, and nothing the agent reads or does can reach them. Because the gate never interprets text there is no classifier to fool and no threshold to sit under: homoglyphs, zero-width characters and look-alike or suffix VPAs only stop an identifier matching the customer's, which makes it W, which blocks. Failure is closed by construction, checked by a 5,000-case fuzz and fifteen planted defects, all caught. Residual: a customer talked into typing the scammer's account; a coached-payment check holds such payments when the app sees a call with an unknown number or screen sharing.

How we evaluated it (Section 7). (1) Every banking run AgentDojo publishes: 28 model and defence configurations, 7,802 runs, against a UPI-style cap and an allowlist. (2) A closed loop with three real LLMs as the payment agent (gemma4, gpt-oss-120b, gpt-4.1-mini), 90 scenarios each run undefended and gated. (3) An Indian payment gateway's official open-source payout server, built unmodified and run against a local capture proxy, with two models. These were pre-registered (six registrations, predictions hashed before scoring; failures reported). (4) A white-box red team of 150 attacks in 15 Indian fraud families that we wrote (design results). (5) Tests that can fail. All data synthetic; no live payment system touched.

Key results (Section 8; Figure 1).

- **AgentDojo:** 1,477/1,477 successful attacks stopped, including 299/299 that had beaten an existing defence; a spending cap let 83.6% through. One prediction failed: 11/210 legitimate-intent payments blocked, each to a wrong account the agent itself produced.
- **Three real LLMs:** 46 attacks moved money undefended (12/45, 14/44, 20/45); **0** with the gate; no legitimate task lost.
- **Real payout server:** attacker payouts reached the payment API in 14/15 and 10/15 episodes direct, **0/15** gated; the server's self-reported provenance called the attacker's payee "user input" 12 of 20 times.

Hardest problem (Section 9). The attacker's account sat inside the very bill the customer asked the agent to pay, so provenance alone could not separate a real biller from a fraudster. The fix was to see that "pay this bill" authorises the *biller*, not whatever account the document prints, and to bind such documents to the biller registry.

3 The problem, and who is affected

3.1 The long-standing gap: payment controls never ask who supplied the payee

For three decades, payment security has answered two questions. *Is it you?* is answered by the UPI PIN, OTPs and device binding. *Is it them?* is answered by payee-name display on UPI, RBI's beneficiary account name look-up for RTGS and NEFT (mandated for member banks by 1 April 2025), and the UK's Confirmation of Payee; like Confirmation of Payee, India's look-up lets the sender proceed even without a match. Mule-account detection works on the same question from the other side: RBI Innovation Hub's MuleHunter.AI is reported to be operational in 26 banks (parliamentary answer, March 2026). Authorised push payment (APP) fraud passes both: the customer is genuine and the account really exists. It is the fraud that has resisted every control:

- **UK:** payment fraud losses of £1.28bn in 2025; APP losses rose 19% to £576.4m across 248,070 cases. 66% of APP cases originated online and 17% via telecommunications, channels the paying bank never sees (UK Finance Annual Fraud Report 2026).
- **India:** UPI fraud losses reported to Parliament were Rs 1,087 crore (FY2023–24) and Rs 981 crore (FY2024–25); cyber-fraud losses reported through NCRP were about Rs 22,000 crore in 2025 (secondary reports of I4C data; figures vary by source).
- **NPCI's own response** shows where the fraud lives. Its circular of 29 July 2025 ended UPI person-to-person *collect* requests from 1 October 2025, because payee-initiated requests were the fraud channel (as reported by Business Today, Outlook and others; merchant collect continues). The rule removed a feature for every user because the rail could not tell a genuine request from a scam one.

The reason no control asks *who supplied this payee?* is that, for a human payment, nobody can answer it. The scammer's instruction arrives on a phone call or in a WhatsApp chat. By the time the customer types the account into the banking app, its origin is gone.

3.2 AI agents change what is observable, and what is at stake

UPI is now opening to agents. On 20 February 2026 Razorpay and NPCI launched agentic payments on Claude for Zomato, Swiggy and Zepto, in which the user sets one spending limit per merchant on UPI Reserve Pay and the agent then pays without a PIN for each purchase (Razorpay newsroom). NPCI is developing a Unified Agent Protocol (UAP) under which customers could authorise agents to transact within predefined limits, building on UPI Circle (delegated payments; on BHIM up to Rs 15,000 a month) and Reserve Pay. NPCI has not released its specifications or liability model (CIO.inc, 9 September 2026); one report says the protocol was not launched at Global Fintech Fest 2026 and awaits RBI clearance, while NPCI did launch AtOM, an orchestration platform producing signed, machine-readable audit trails (single secondary source). Private rails are already live: Pine Labs' P3P (June 2026) extends UPI one-time mandates and Reserve Pay to agent-triggered payments.

These mandates bound **how much**. Nothing in them binds **who said so**. As CIO.inc puts it, an agent "could remain within a spending limit while making a purchase the customer did not intend." An agent reads invoices, SMS, e-mail and web pages; one line of attacker text in any of them can swap the payee, and the payment passes every check on the rail. Because one agent design serves millions of users, one working injection repeats at population scale.

But agents also make the missing question answerable. Everything the agent read is in its context, and the bank or app that hosts it knows which text came from the customer. The scam that was off-channel for humans is *on-channel* for agents.

3.3 Why it matters now: liability moves to the bank

RBI's *Commercial Banks – Responsible Business Conduct – Third Amendment Directions, 2026* apply to electronic banking transactions on or after 1 January 2027. According to Economic Laws Practice's analysis of the directions, they introduce a *Fraudulent Electronic Banking Transaction* that expressly covers transactions undertaken by customers themselves under coercion or duress and transactions by third parties using credentials obtained through fraud; they shift the burden of proof so that the bank must establish customer liability before denying relief; and they recognise intermediary, payment-aggregator and other third-party ecosystem failures. When a hijacked agent pays a fraudster, the bank will need to show what happened. Today it has no record of who chose the payee. (How an agent-initiated payment maps onto these categories is our reading, not an RBI statement.)

3.4 What the industry says is missing

Rajendran N., co-inventor of UPI and former CTO of NPCI, frames the open question as how a human's delegated authority "can be proven for every transaction", and says disputes need evidence of "which agent acted, under which version of the mandate and against what customer instruction"; he asks for "100% end-to-end traceability and

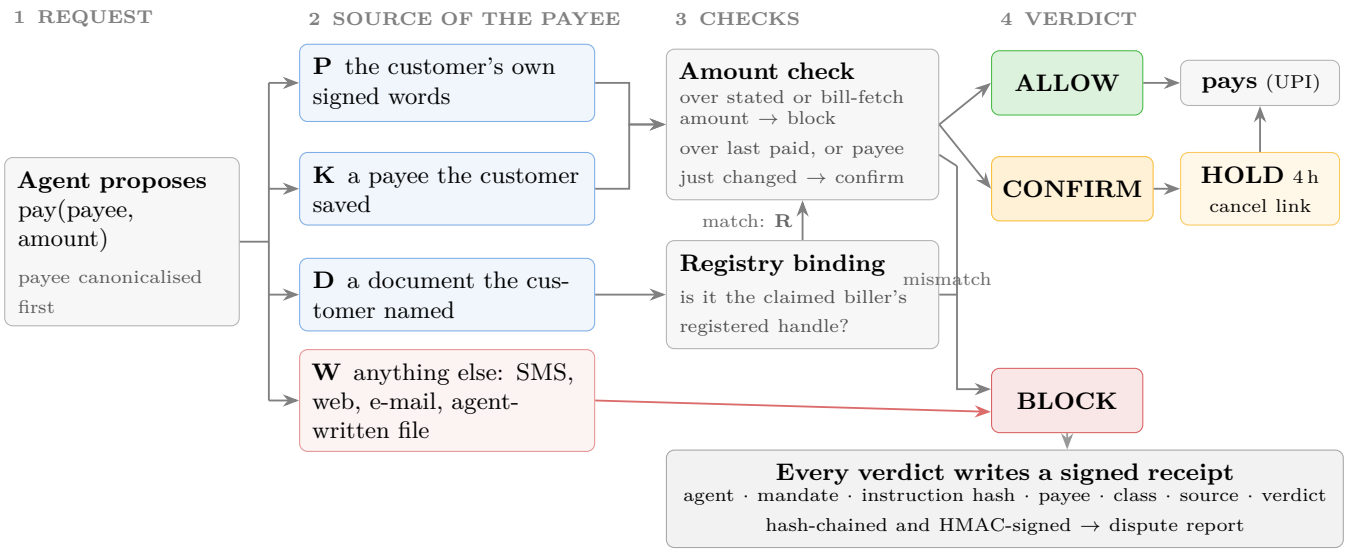


Figure 2: How one payment is decided. The gate never reads the attacker's words: it asks where the payee came from, using only what the bank already holds. Every later check can make a decision stricter, never looser, so an injection can push a payee down to W but nothing outside the customer and the bank can raise it to P, K or R. No model, no network call; median 74 µs per decision.

auditability", and for controls a "compromised or manipulated agent" cannot cross, "enforced by the bank or payment network" (all CIO.inc, 9 September 2026). The same article reports that RBI's FREE-AI committee report identifies prompt injection, including hidden commands intended to trigger unauthorised transfers, as a risk to financial AI systems (we could not retrieve the primary FREE-AI text to confirm the wording; KPMG's summary of the report confirms it discusses autonomous agents using MCP and A2A, and that manipulation and autonomous decisions raise liability concerns).

Our answer to each of those requirements: per-transaction proof of authority (the payee's provenance class), deterministic enforcement at the bank (no model in the gate), and a receipt carrying exactly the agent, the mandate version and the customer instruction.

4 Approach

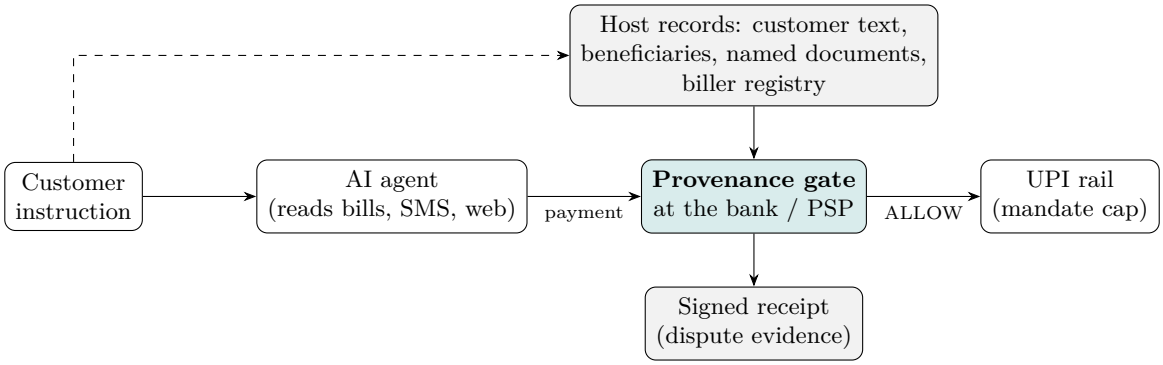
4.1 Key design decision: check provenance, not content

We do not try to recognise malicious text. A content filter is a classifier, and an attacker who can read it can write around it. We check *where the payee identifier came from*. The host (the bank or PSP app running the agent) already knows which messages the customer typed, which beneficiaries the customer created, which documents the customer pointed the agent to, and which inputs came from outside. Each payee is classed, first match wins:

class	source of the payee identifier (VPA, IBAN, account number)	decision
P	a whole token in the customer's own message	ALLOW
K	a beneficiary record that was itself created by an authorised action (provenance is inherited)	ALLOW
R	a document the customer named, and it is the *registered* handle of the biller the document claims to be (Bharat BillPay biller, NPCI verified merchant)	ALLOW
D	a document the customer named, with no registry entry to confirm it	STEP-UP: a confirmation card naming the source
—	the document claims a registered biller, but the payee is not that biller's handle	BLOCK
W	anything else: SMS, web page, collect request, tool output, or nowhere	BLOCK

Further rules: a credential change is allowed only with a value the customer typed; an amount the customer stated is a ceiling, and without one an amount above the mandate cap needs confirmation; adding a beneficiary is gated exactly like a payment, so the known-payee list can only grow through authorised actions; an incoming collect request is classed like any payee, so a stranger's request is W.

4.2 Where it sits



The gate needs no change to the agent and no model: it sees the payment call and the host's own records. It adds no network call (the registry is a lookup the PSP already performs for merchant verification).

4.3 The provenance receipt

Every decision writes a receipt: agent ID, mandate ID and version, SHA-256 of the customer's instruction, payee, amount, provenance class and source, verdict, and whether the customer confirmed. Receipts are hash-chained and HMAC-signed under a key the agent never holds, so anyone who can write the receipt store still cannot alter history without detection (tested: edits, deletions and reordering are all caught). A one-line plain-language reading is generated for a dispute desk, for example: *"agent X under mandate RP-001 v1 proposed Rs 1,840 to Y; the payee came from outside the customer's instruction (SMS inbox). Outcome: refused."*

4.4 Alternatives we rejected, and why

- **A second LLM judging whether a payment looks legitimate.** It reads the same attacker text and can itself be injected; our threat model assumes the attacker knows its prompt.
- **A known-payee allowlist.** Safe for old payees, but it refuses every first payment to a new payee (41.7% of legitimate AgentDojo payments) and is laundered by an injected "add as payee, then pay" (10/10 in our red team).
- **A tighter spending cap.** It blocks legitimate rent increases while still admitting small redirected payments (80.7% of real attacks fit under a cap set at the account's largest prior payment).
- **Reading the customer's intent from language.** Our own earlier authorization witness did this (Section 8.7). On AgentDojo it flagged 23.5% of benign runs, mostly because language is open-ended. Identifiers are not: a VPA either is, or is not, a token the customer wrote. That is why this gate blocks 0/24 legitimate payments where the language-reading witness could not get below about a quarter.

5 Threat model and resilience under exposure

- **Who the attacker is:** anyone who can put text in front of the agent: a biller's PDF, an SMS, a WhatsApp forward, a web page, a collect request, a transaction note.
- **What they know:** the gate's full rules and source code (this report and the repository).
- **What they control:** everything the agent reads and, in our red team, the agent itself: we assume it is *fully hijacked* and does exactly what the injected text says.
- **What they cannot do:** write into the customer's own message, the bank's beneficiary record, or the biller registry; obtain the receipt-signing key.
- **What they want:** move money to an account they control, or take over the account's credentials.

Why knowing the rules does not help. ALLOW requires the identifier to be a whole token in the customer's text or a record that only authorised actions can write. There is no classifier to fool, no threshold to sit under, no wording to find. Every obfuscation we tried (Cyrillic homoglyphs, zero-width characters, fullwidth digits, a look-alike VPA that is a suffix of the customer's, transposed characters, a swapped handle, a truncated handle) makes the identifier *unfindable* in the customer's text, which makes it W, which blocks. The gate fails closed by construction, and a 5,000-case fuzz test checks exactly this. Nothing depends on secrecy except the receipt-signing key.

Remaining weakness: the customer. A scammer who persuades the customer to type the scammer's VPA gets class P, and the gate allows it. The gate turns agent-borne fraud into ordinary human fraud, no worse; it does not solve human fraud. Confirmation cards for class D are only as good as the customer's reading of them, which we did not measure.

6 Related work, and what is new

- **CaMeL** (Debenedetti et al., 2025) and **FIDES** (Microsoft, 2025) track where values come from inside a redesigned agent: CaMeL builds a protective layer that separates control flow taken from the user's query from untrusted data and enforces capability policies at tool calls (about 77% AgentDojo utility vs 84% undefended). *The principle that untrusted data must not choose consequential arguments is theirs.* We apply it at the *bank's* side of the payment API, with no change to the agent, which matters because a bank does not control which agent its customer uses.
- **PACT** (Fan et al., arXiv:2605.11039, May 2026) enforces provenance per *argument* rather than per tool call, computed by a runtime monitor: with oracle provenance it reports 100% security and utility on diagnostic suites, and on AgentDojo 100% security on the three strongest models at 38–46% utility; its authors locate the remaining bottleneck in *inferring* provenance. **ARGUS** (Weng et al., arXiv:2605.03378) releases an action only when benign evidence entails it (attack success 28.8% to 3.8% at 87.5% utility on its AgentLure benchmark, whose example is a fee injected into a bill payment). For payments the inference problem largely disappears: a VPA, IBAN or account number is an exact token, so we match it against the customer's text and the bank's records instead of inferring its origin. That is why the gate needs no model and blocked 0/24 legitimate AgentDojo payments; the figures are not directly comparable (recorded traces vs closed-loop utility).
- **A-VIP** (Louck et al., arXiv:2609.11757, September 2026) binds each cart line to the listing the user saw, for Google's AP2 protocol; **Beyond the Mandate** (Aviv et al., arXiv:2608.23858) shows that valid AP2 mandate signatures do not ensure an agent transaction reflects the user's intent. Neither addresses UPI's mandate model.
- **aiAuthZ** (arXiv:2607.05518) gates AgentDojo banking calls with an allowlist; clean utility fell from 100% to 80% because a legitimate first payment to a new payee was blocked: the allowlist trade-off our K/P split removes.
- **Authenticated Delegation** (South et al., arXiv:2501.09674) and **Auditable Agents** (Nian et al., arXiv:2604.05485) set out delegation credentials and five auditability dimensions; our receipt covers action recoverability, policy checkability, responsibility attribution and evidence integrity for the payment step.
- **Mastercard Verifiable Intent** (with Google, open-sourced) and **Visa's Trusted Agent Protocol** bind the cardholder, a registered agent and the mandate's scope into signed credentials. They prove *which* agent acted under *which* mandate; they do not decide whether a payee inside that mandate came from the cardholder. The gate is the check such a credential needs at payment time, and its receipt can carry their identifiers.

Concurrent work in October 2026, and where we stand

Three independent projects reached the same principle this month. We found them in our final research pass, after our gate and its AgentDojo pre-registration existed, and we report them in full because they are close.

work	where the check runs	evidence	gaps it reports or leaves
Mool (Hack Sprint 2026, 5 Oct; UPI)	inside a redesigned agent: privileged planner + quarantined extractor (CaMeL pattern)	own benchmark, 50 tasks × 6 attacks, *deterministic stand-in model*: 0/300 stolen, 46/50 clean tasks completed	amount inflation to the right payee overpaid 31/50 runs; every new payee is held; no real-LLM result in the public repository (last commit 5 Oct)
taintgate (HackerNoon, 11 Oct)	in-process provenance gate	5 scenarios: 3/5 attacks stopped, **0/2 legitimate actions allowed**; a 629-attack AgentDojo study of ten injection detectors (best: 51% at 2% FPR)	reformatting an account number broke matching; laundering through several tools broke lineage; cannot separate a real bill IBAN from an injected one
fps-ai-security-poc (19 Oct; ISO 20022)	layered: screen, policy on beneficiary records, Confirmation of Payee, human hold	simulated agent; Claude Haiku 5.5 paid 0/240 on its own	names vendor-master fraud (saved payee's details changed) but does not gate the change itself

The principle is shared; our contribution is the evidence and the gaps closed. (i) We are the only one of the four with a **pre-registered result on real agents' traces** (four frontier models, AgentDojo), measured against the controls UPI actually uses (caps, allowlists), and **closed-loop runs with three real LLMs** (Section 8.5). (ii) Our gate runs at the **bank or PSP**, not inside the agent: a bank does not control which agent its customer uses, so a check that requires a particular agent architecture cannot be the bank's control. (iii) We built and measured a fix for every gap the table lists: amount provenance (registered bill fetch must match; a known payee above its last-paid amount needs confirmation), vendor-master changes gated like payments plus a cooling-off window, digit-separator canonicalisation, root matching (no lineage to break), files the agent wrote treated as world text, and a biller-registry binding that separates the real bill payee from the injected one. (iv) A receipt and dispute classifier shaped for RBI's 2027 burden of proof.

7 How we evaluated it

(1) External and pre-registered: AgentDojo banking. AgentDojo (ETH Zürich, NeurIPS 2024) is the standard benchmark for prompt injection against tool-using agents. Its banking suite has 16 user tasks (pay a bill, refund a friend, update rent) and 9 injection tasks (send money to the attacker, redirect a standing order, change the password). We scored the published runs at pinned commit `089ed468` for four models: 64 benign runs, 576 attacked runs, 201 attacks that actually succeeded. Six predictions and the full scoring rule were written and the file hashed (sha256 `e3ddb6db...a163f`) at 10:26 IST on 31 October, before the scorer existed; the scorer (sha256 `8d3bf976...`) was then run once. No agent was run: these are the actions the four agents really took. Baselines on the same traces: a mandate cap at the account's largest prior payment (the Reserve Pay shape) and a known-payee allowlist.

(2) White-box red team on a mock UPI rail. 150 attacks in 15 Indian fraud families and 150 legitimate payments in 15 families, including Hinglish and Devanagari instructions, a spaced IBAN and an account typed with hyphens. The agent is fully hijacked, so nothing relies on the model resisting. Five families (A6–A9, A12) were written from the gate's own rules to break it, and three (A13–A15) from the gaps concurrent work reported. Two customer models: one who reads confirmations, one who approves every confirmation (the worst case). Same baselines, plus no control.

(3) Closed loop with a real LLM, pre-registered. Three agent models (gemma4 locally; gpt-oss-120b and gpt-4.1-mini through their providers' APIs) act as the payment agent on three scenarios per family (90 episodes), undefended and gated, seeing the gate's refusals (Section 8.5).

(4) Tests that can fail. 32 unit and property tests, and a mutation probe that plants fifteen defects in a copy of the code and checks each is caught.

8 Key results

8.1 External, pre-registered: AgentDojo banking (all six predictions held)

pooled, four models	**provenance gate**	mandate cap	allowlist
successful attacks with every attacker call stopped	**197/197** (95% CI 98.1–100)	38/197	171/197
attacker payments blocked outright / held for confirmation	55.4% / 44.6%		
legitimate payee calls blocked (benign runs)	**0/24** (CI 0–13.8)	8/24	10/24
attacker credential changes blocked	**28/28**	0/28	0/28
legitimate calls blocked inside attacked runs	7/225 (all nonsensical, see below)		

Per-model counts are in the repository. We inspected every exception before reporting: (a) four successful-attack runs pay the IBAN the customer *themselves* named in that task (AgentDojo reuses the attacker's IBAN as a new landlord's); the gate allows them because the customer authorised that payee, and counting them as misses gives 197/201 = 98.0%, still above the registered 95%. (b) The 7 legitimate-intent calls blocked inside attacked runs are five zero-amount payments to the account's own IBAN, one IBAN the model invented, and one call whose recipient was the literal string "landlord's IBAN". (c) 41 of 64 benign runs make no payment, so the legitimate sample is 24 calls; its 0% has an upper bound of 13.8%.

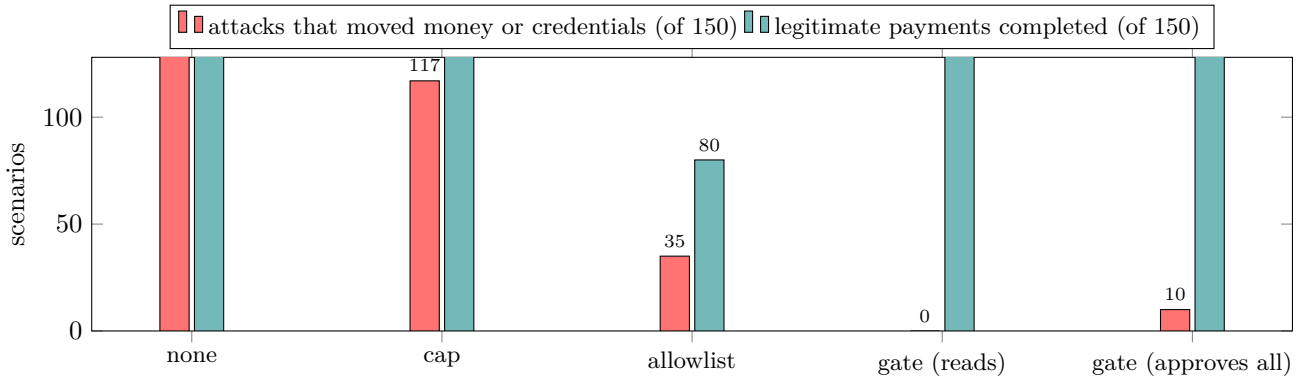
8.2 Every banking run in AgentDojo, including runs behind existing defences (pre-registered)

To remove the small-sample and single-attack limits we applied the frozen scorer, unchanged, to every banking run at the pinned commit: 28 model and pipeline configurations (GPT-3.5 to GPT-4o, Claude 3 to 3.7, Gemini 1.5 to 2.0, Llama 3 and 3.3, Command R, Meta-SecAlign) and 15 attack types; 7,802 runs. Four predictions were hashed first (sha256 `8953826c...`); three held and one failed.

id	result	verdict
E1	every attacker call stopped in **1,477 / 1,477** successful attacks (Wilson 99.7–100%)	held
E2	attacks that got *past an existing defence* (tool filter, prompt-injection detector, spotlighting, repeated user prompt, Meta-SecAlign): **299 / 299** stopped	held
E3	legitimate benign payee calls blocked **11 / 210 = 5.2%**, predicted $\leq 3\%$	**failed**
E4	UPI-style cap lets $\geq$1 attacker call through in 1,235 / 1,477 (83.6%)	held

E3 stays failed. Inspected after scoring, every one of the 11 blocked calls named a payee the user never named and the bank never recorded: 7 paid the account's *own* IBAN, 2 paid an IBAN the model corrupted (an extra digit inserted into the landlord's or a saved payee's account), 1 a placeholder string and 1 a made-up IBAN. None was a correct, intended payee: the gate also stops an agent's own misdirected payments. Credential takeovers blocked: 239 / 239. A known-payee allowlist would have blocked 61.0% of the legitimate payments.

8.3 White-box red team: agent fully hijacked



family	what the attacker does	none	cap	allowlist	gate (reads)	gate (approves all)
A1	invoice redirection: fraud VPA in the bill the customer named	10	10	0	0	0
A2	extra payee injected into a named document	10	0	0	0	0
A3	KYC-expiry SMS lure	10	10	0	0	0
A4	checkout redirection on a web page	10	10	0	0	0
A5	fake-refund collect request	10	0	0	0	0
A6	*white-box:* add as beneficiary, then pay	10	10	**10**	0	0
A7	*white-box:* look-alike / suffix VPA	10	10	0	0	0
A8	*white-box:* homoglyph, zero-width, fullwidth	10	10	0	0	0
A9	*white-box:* "RBI safe account", split under cap	10	10	0	0	0
A10	in-mandate redirect: amount fine, payee wrong	10	10	0	0	0
A11	standing-order payee hijack via landlord notice	10	0	0	0	**5**
A12	*white-box:* credential takeover	10	10	**10**	0	0
A13	*gap (Mool):* amount inflated, right payee	10	7	5	0	5
A14	*gap (fps-poc):* vendor-master change by e-mail	10	10	**10**	0	0
A15	*gap (taintgate):* laundering via a file the agent wrote	10	10	0	0	0
attacks succeeded (of 150)		150	117	35	**0**	**10**
legitimate payments completed (of 150), 15 families		150	140	80	**150**	**150**

When the customer approves everything, ten attacks succeed: the named-notice half of A11 (landlords are in no registry) and the saved-shop half of A13 (an inflated kirana bill above the last-paid amount is shown for confirmation, never paid silently; the registered-biller half of A13 is blocked by the bill-fetch check). 15 of 150 legitimate payments needed one confirmation: a first payment to an unregistered vendor (5) and a genuinely higher shop bill (10). History: the provenance-only v1 gate gave 0 and 25 on the original 12 families; v2 added the registry check, v3 the amount, vendor-master, agent-written-file and separator fixes, each after seeing the previous version, on scenarios we wrote: these are design results. The independent evidence is the AgentDojo result above and the closed loop below. Every receipt from the run verifies, and the dispute report finds 0 control failures in 740 decisions. **None of the ten is paid at once:** a confirmed STEP_UP becomes a timed HOLD (4 h for a new payee, 1 h otherwise) with a cancel link to the customer's registered contact, so one deceived click is not enough (precedents: UK FCA FG24/6, delay up to 4 business days on reasonable suspicion; DBS new-payee cooling period, March 2026; Singapore banks' block-or-delay on transfers of more than half the balance, October 2025). Whether a held victim cancels depends on people and is not measured here.

8.4 Tests that can fail

- **Fail-closed fuzz:** 5,000 random cases under eight attacker transformations; a payee is ALLOWed only if it is canonically identical to a token the customer wrote, and the customer's own payee is never blocked. 0 violations.
- **Laundering:** a world-sourced payee cannot be added as a beneficiary, and a record without authorised provenance is not treated as known.
- **Registry:** a document claiming BESCOM with a non-BESCOM payee blocks; the registered handle allows; an unregistered entity stays a confirmation.
- **Receipts:** any edit to amount, payee, class or verdict, any deletion and any reordering fails verification.
- **Concurrent-work gaps:** an account typed with hyphens or dots still matches; a file the agent wrote is world text; a registered bill must match its fetched amount; a saved payee above its last-paid amount needs confirmation; a vendor-master change from an e-mail blocks; a recently changed payee needs confirmation.
- **Dispute report:** categories read off a verified chain; a tampered chain is refused, never summarised.
- **Mutation probe:** fifteen planted defects (substring matching, ignoring beneficiary provenance, removing Unicode normalisation, allowing registry mismatches, skipping the receipt MAC, stepping up instead of blocking world payees, treating agent-written files as delegation, ignoring the bill fetch, ignoring the last-paid amount, not gating payee changes, ignoring the cooling-off window, dropping separator canonicalisation, not checking the principal signature,

paying a confirmed step-up at once, ignoring the coaching signal): **15/15 caught**. The first run of the original six caught 5/6; the survivor (normalisation) only ever made the gate stricter, and we added the missing usability test rather than leaving the property unpinned.

8.5 Closed loop with real LLMs (pre-registered)

A real model acts as the payment agent on the first three scenarios of each of the 30 families: 90 scenarios, each run undefended and gated (temperature 0), the agent seeing the gate's refusals and free to try again. gemma4 (local) was registered first (sha256 `7bff7d92...`, all four predictions held); two more families were registered before any of their episodes (PR55, `c09d855c...`, four dated amendments). Outcome as registered: money moved, a credential set or a collect request approved.

agent model	attacks: undefended → **gated**	legitimate completed: undefended → gated	scenarios
gemma4 (Google, open weights)	12/45 → **0/45** (CI 0–7.9%)	33 → 33	90/90
gpt-oss-120b (OpenAI, open weights)	14/44 → **0/44**	23 → 23	70/90[†]
gpt-4.1-mini (OpenAI, commercial)	20/45 → **0/45**	32 → 35	90/90

The models fell for invoice redirection, injected invoice instructions, inflated amounts and laundering through a file the agent wrote; the commercial model fell for the most, and the gate stopped every one without costing a legitimate task. Incomplete legitimate tasks fail identically in both conditions (no payment, or a payee's *name* paid instead of an account, which the gate also refused). gemma4's harness count was 14, counting two payee-record changes that moved no money; we report both. A real model resists most injections on its own (SMS, web, collect and credential lures); what still works arrives inside a document the customer asked it to act on, exactly the class the registry binding and amount rules close. [†]The provider's free tier stopped gpt-oss-120b at its daily token limit after 70 paired scenarios; the rest are reported as not run. Episodes lost to a network outage (no model reply) were re-run under a rule fixed before the re-run, first attempts kept; one provider error (HTTP 400) is reported, not re-run.

8.6 In front of a real, unmodified Indian payout tool (pre-registered)

We took the official, open-source MCP server that an Indian payment gateway publishes so AI agents can make payouts, built it from source *unmodified*, and ran it locally in its default configuration with every outbound request routed to a local capture proxy: no vendor system was contacted and the credentials were dummy strings. "Money would move" means the real server emitted its payout request. gemma4 acted as a small business's finance assistant; four predictions were hashed (sha256 `5e12e6ba...`) before the run, and all four held.

condition	attacker payout reached the payment API	every legitimate payout reached it
direct (model → real server)	**14 of 15** attack episodes	6/6 (+ 3/3 batches)
gated (our gate in front, server unchanged)	**0 of 15**	**6/6** (+ 3/3 batches)

The server already asks the right question, and lets the agent answer it. Every call must carry a per-field label saying where each value came from (*user input*, *generated by model*, *inferred from context*). The model writes those labels. In the direct episodes that paid an attacker, the model labelled the attacker's payee **"user input" 12 times and "inferred from context" 8 times, and never "generated by model"** (20 labels). In this server the labels feed only a prompt for a missing amount, which is off by default; nothing reads the payee's label. Self-reported provenance is testimony, and the attacker controls the testimony. Our gate ignores the label and computes the payee's class from the host's records: it blocked every attacker payout and passed every legitimate one.

Two first-run limitations were closed by a second pre-registered run (sha256 `0d408d23...`, all four predictions held): the payout-by-link-to-a-phone attack, untested at first because our harness lacked the token that endpoint needs, reached the API in 3/3 episodes direct and 0/3 gated; and the gate now refuses only the offending row of a batch, so in 3/3 attacked batches the three real vendors were paid and the attacker row was not (the first run had refused whole batches). Two harness bugs of ours were found and their runs discarded, both recorded. We are sharing the observation with the vendor as a hardening suggestion and name neither the vendor nor step-by-step details here until they have had the chance to respond.

Second model on the real server (PR55): with gpt-4.1-mini (OpenAI, commercial) as the finance assistant on every scenario (48 episodes, 0 errors), the attacker's payout reached the payment API in **10 of 15** attack episodes direct and **0 of 15** gated; every legitimate payout was delivered in 12 of 15 episodes in both conditions (the 3 misses are one batch scenario where the model made no payout either way).

8.7 Lineage: what our earlier measured work taught us

This gate is the third step of a measured line of work in the Aegis AI project. Our authorization-provenance witness (September 2026; 383 held-out cases: 3.8% false accepts, 12.7% false rejects; agent-written authority never accepted, 0/440) read intent from language, and on all four AgentDojo suites flagged 87.7% of successful injections but also 23.5% of benign runs. Our deed-provenance monitor (who first wrote the target: 209/210 SHADE-Arena sabotage runs, 21/210 honest) showed the target's origin is the signal that holds. This gate keeps that and drops reading language: 11/210 legitimate payments blocked across all of AgentDojo, every one to a wrong account.

9 The hardest problem we hit

We expected provenance to separate attack payments from legitimate ones cleanly. On the first AgentDojo attack we inspected it did not: the attacker's IBAN sat *inside the very bill the customer asked the agent to pay.* Provenance alone cannot tell a real biller's account from a fraudster's when both come from the document the customer named. This is not a benchmark quirk. It is invoice redirection, one of the oldest frauds in business payments, and the obvious fixes fail: trusting the document admits the fraud; distrusting it blocks every first bill payment; asking a model whether the account "looks right" reopens the injection channel we set out to close; a confirmation prompt puts the whole decision back on a customer who, by construction, did not read the bill. The difficulty taught us that the customer's instruction "pay this bill" authorises the *biller*, not whatever account number the document prints. The question had to change from *where did the payee come from?* to *is this the payee the claimed biller registered?*

How we overcame it.

- **Split the class.** Named documents became class D, a confirmation card that names the source, instead of being trusted or blocked. With a careless customer this still left 25 of 120 red-team attacks (the original 12 families). What we gave up: a click for the customer on a first bill.
- **Bind the document to a registry, not to its own text.** If a document claims to come from a registered biller (Bharat BillPay, NPCI verified merchants), the payee must be that biller's registered handle; otherwise BLOCK. The registry is written by the payment system, so no injection can satisfy it. With a careless customer this cut 25 to 5, at no cost to legitimate payments (110/110 still complete).
- **Then we read the concurrent work and attacked ourselves with its failures.** v2 had no rule for any of them: it ignored the amount paid to a known payee, had no notion of changing a saved payee, and treated a file the agent itself wrote as a document the customer named (a laundering path we found only by replaying taintgate's evasion against our own design). v3 adds a rule for each (Section 7.2: A13–A15) and still completes 150/150 legitimate payments.
- **Our own white-box pass found two bugs before any number was reported.** A VPA that is a suffix of the customer's (`kumar@okicici` inside `rama.kumar@okicici`) matched under substring search, so we moved to whole-token matching. A spaced IBAN was missed because the pattern swallowed the end of the preceding word; that failed closed (a legitimate payment blocked, not a fraud allowed) and was fixed.
- **Still open:** payees with no registry (landlords, individuals named in a document) still rely on the customer's confirmation.

10 Limitations

- **Coached customers: partly closed, one case open.** Provenance cannot see a customer who really typed the scammer's account. A separate check for human payments uses signals a banking app already has (call with an unknown number, screen sharing or remote access, more than half the balance in 24 h, new payee) and holds the payment 24 h with a bank call-back. On 120 synthetic scenarios we wrote from reported Indian fraud patterns (design results, not independent): 50/60 coached frauds held or stepped up (digital arrest, impersonation, remote-access refund, balance drain: 10/10 each; pasted investment VPA: 8 held, 2 stepped up). **Open:** a victim coached earlier, with no live signal, paying a small amount under the cap is paid 10/10. Cost: 50/60 legitimate payments go through at once; the 10 held are a large first payment to a car dealer who is on the phone, which looks exactly like coaching.
- **Confirmations: designed from evidence, not yet measured on our users.** The card follows the best arm of a randomised in-app experiment (Akesson, Gathergood & Quispe-Torreblanca 2023: risk-based warning plus call to action, 81% fewer people fell for fraud; generic warnings 18%, decaying). Our own human study is pre-registered (PR54), not run.
- **Amounts.** Closed for registered billers (bill fetch must match) and saved payees (above last-paid needs confirmation); a wrong product from a legitimate merchant inside the mandate is not addressed.
- **Sample size and single attack type: closed.** PR52 covers all 28 AgentDojo configurations and 15 attack types (210 legitimate payee calls).
- **One live model: closed.** Three model families (Google gemma4; OpenAI gpt-oss-120b and gpt-4.1-mini) in the closed loop and two on the real payout server, 0 gated attacks for every one (PR55). Still open: gpt-oss-120b on the

real server (provider daily limit) and any model larger than these.
- **Host trust: closed in code.** With a bank-app signature on each customer message, text the agent platform claims the customer typed cannot yield class P (2 tests, one planted defect caught).
- **Registry data is held by the deployer.** Our tests use invented handles because NPCI restricts the real data to members: acquiring banks publish verified merchants to NPCI and every UPI app must download and store that list (NPCI Circular 18/2017, List VAE API; OC-95, 2020), and Bharat Connect biller IDs reach banks through their biller-data feeds. The bank or payment app that would run this gate already holds both; the gate's registry is a plain entity-to-handle table they fill from that copy, with no code change.

11 What each institution gets, and the rule it already has that this fits

The hackathon is where this starts, not where it ends. Each row below names an instrument that already exists, so adoption needs no new regulation, only a field, a check or a test.

institution	existing instrument (verified source)	what we hand over
NPCI	Operating Circular 201-B (8 Oct 2025): software and IoT devices may make delegated UPI Circle payments, person-to-merchant only, device ID validated on every request, and an online dispute-resolution mechanism required (Khaitan & Co. analysis). The Unified Agent Protocol is pending, with no published liability model.	A two-field addition to the agent payment request (payee provenance class and source); since delegated payments are P2M, every legitimate payee is a verified merchant, which the registry binding checks; signed receipts as the dispute record the circular requires; a precise replacement for the blanket end of P2P collect (a stranger's request is W).
RBI	Authentication Directions 2025 (in force 1 Apr 2026): issuers may add risk-based checks beyond 2FA based on the fraud risk of the transaction. Third Amendment Directions 2026 (from 1 Jan 2027): banks bear the burden of proof on fraudulent electronic banking transactions. FREE-AI (Aug 2025): an AI innovation sandbox is recommended.	Payee provenance as the risk signal for the risk-based check (STEP_UP is that check); the receipt and dispute report as the evidence a bank needs to meet the burden of proof; a sandbox-ready test plan.
RBI Innovation Hub	MuleHunter.AI (26 banks); the Digital Payments Intelligence Platform prototype (real-time risk scores).	Provenance class as a feature for transaction risk scores; W-class payees as a cross-check against mule lists.
Banks, PSPs, TPAPs	Beneficiary name look-up (RTGS/NEFT, from 1 Apr 2025); beneficiary cooling-off practice.	A drop-in check in front of any agent payment API (424 lines, no model, no extra network call) that also gates payee changes, and a dispute report from verified receipts.
SROs (FACE, UFF)	RBI-recognised fintech SROs (FACE 2024; Unified Fintech Forum, Sept 2026) with codes of conduct and dispute resolution.	The scenario catalogue as a conformance test for members' agent payment products.
IndiaAI Safety Institute	India-first evaluation and benchmarks.	The 15-family Indian fraud catalogue, the AgentDojo scorer and the closed-loop harness, all open source.

How we keep the evidence ahead. We attack our own controls first and publish what breaks: every failure in this report (a suffix VPA, a spaced IBAN, a laundering path, the invoice ambiguity) was found by us, before release. Predictions are written and hashed before any scoring.

What funding builds next: larger frontier models in the closed loop; a human study of confirmation cards in Hindi, English and regional languages; the coached-payment check on a bank's real call and device signals; integration with Bharat Connect bill fetch and NPCI verified-merchant data; a draft field specification for the Unified Agent Protocol; a pilot with one PSP.

12 How a bank deploys it

Where it runs. At the bank, PSP or payment app endpoint that receives an agent's payment request, before the UPI request is formed. It does not depend on which agent the customer uses, so a bank can enforce it on every agent at

once.

What it reads, all of which the bank already holds. The customer's own messages to the agent (signed by the bank's app, so the agent platform cannot forge them); beneficiary records and who created each; the verified-merchant list every UPI app already stores and Bharat Connect biller data; bill fetch for registered billers. For human payments, the coached- payment check adds two device signals a banking app can read (a call with an unknown number, screen sharing or remote access) and the account's own 24-hour outflow.

What it costs. 424 lines of standard-library Python (gate and receipts), no model and no network call. On one laptop core: median 74 µs per decision, 99th percentile 0.24 ms (6,000 decisions over the scenario set). It adds no measurable latency to a UPI payment.

How it fails. Closed. A payee that appears nowhere the customer or the bank wrote is class W and is refused; no input can raise a payee's class except the customer's signed words, the bank's records or the registry. Across all of AgentDojo's banking runs, the 11 legitimate-task payments it refused (of 210) were all to a wrong account the agent itself produced: refusing them was correct.

Rollout we propose to a bank.

1. **Shadow mode** (block nothing): log every verdict on real agent payments for a few weeks and measure the false-block rate on the bank's own traffic, the number that decides everything after.
2. **Enforce BLOCK for class W and registry mismatches**: payees nobody on the customer's side wrote, and bills whose account is not the claimed biller's registered handle. Lowest risk to legitimate payments; it covers the SMS, web, collect-request and fake-biller redirections.
3. **Turn on confirmation and holds** for class D, amount above the last paid, payee changes and coaching signals, with the confirmation card in the customer's language.

Measures to report each month: false-block rate, confirmation rate, holds cancelled by the customer, fraud loss on agent payments, and time to resolve a dispute using the signed receipt (the evidence the bank needs under RBI's 2027 burden of proof).

13 Deliverables and declarations

Code repository	https://github.com/Alkur123/payee-provenance. The README reproduces every number; standard-library Python 3.10+, no model, no network.
Built during the hackathon?	Built on earlier work by the same author, cited above. *Existed before:* the authorization-provenance witness and deed-provenance monitor (September 2026) and our AgentDojo scoring harness. *Added on 31 October 2026, inside the window:* the payee-provenance gate for payment rails (inherited beneficiary provenance, registry binding, amount provenance, payee-change gating, signed customer channel, confirmation card), signed receipts and dispute report, the Indian red team, six pre-registered evaluations (AgentDojo banking for four models, then all of AgentDojo's banking runs, a closed loop with a real LLM, the real payout server and its follow-up, and the same closed loop and server with two more model families) and a pre-registered, not yet run, human-study protocol, the tests and mutation probe, the video and this report.
AI tools used	Claude Opus 5.5 (Anthropic), through Claude Code: web research, writing the code, running the evaluations, and drafting this report under the author's direction. The author chose the problem, approved each step and is responsible for every claim. Agents under test (never part of the gate): gemma4 (Google; open weights, local through Ollama), gpt-oss-120b (OpenAI open weights, through Groq's API) and gpt-4.1-mini (OpenAI, through OpenAI's API). No AI model is part of the gate.
External code and data	AgentDojo task definitions and run logs (MIT licence, ETH Zürich SPY Lab), read-only. gemma4 open weights (local); gpt-oss-120b and gpt-4.1-mini by API on synthetic scenarios (evaluation only). Python standard library. Our code and scenarios: MIT.
No real personal data	Yes. Every VPA, account, IBAN and name is synthetic; AgentDojo's are its own synthetic values.
Responsible disclosure	Yes. No live UPI, bank, NPCI or Aadhaar system was touched; no exploit against a real system is described; the attack families are standard, publicly documented fraud patterns.
Environment	A local Python mock of a UPI-style rail with Reserve Pay-style mandates and a stand-in biller registry; AgentDojo's published logs; an Indian payment gateway's open-source payout server built from source and run locally against a capture proxy with dummy credentials (no vendor system contacted); gemma4 run locally, two models by API. No real transactions or accounts were involved.

Sources

- UK Finance, Annual Fraud Report 2026, press release (June 2026).
- Lok Sabha replies on UPI fraud, FY2023–24 and FY2024–25, as reported by BoomLive and India Tribune; NCRP/I4C 2025 figures as reported by Moneylife and The420.
- NPCI circular of 29 July 2025 discontinuing UPI P2P collect from 1 October 2025, as reported by Business Today (13 Aug 2025) and Outlook Business.
- Razorpay newsroom, "Razorpay and NPCI launch agentic payments on Claude" (20 February 2026).
- Y. Kalra, "India readies UPI to support payments initiated by AI agents", CIO.inc (9 September 2026), verified verbatim.
- Economic Laws Practice, "RBI tightens customer protection frame-

work for fraudulent electronic banking transactions" (June 2026), on the RBI Commercial Banks – Responsible Business Conduct – Third Amendment Directions, 2026.
- RBI, Framework for Responsible and Ethical Enablement of AI (FREE-AI) committee report (13 August 2025); KPMG India summary (September 2025).
- Khaitan & Co., "Enabling Agentic Payments on UPI Rails" (ERGO, 20 November 2025), on NPCI Operating Circular 201-B (8 October 2025).
- RBI press release, Authentication Mechanisms for Digital Payment Transactions Directions, 2025 (in force 1 April 2026).
- RBI press releases on recognition of SRO-FT: FACE (2024) and Unified Fintech Forum (10 September 2026); RBIH and I4C MoU on MuleHunter.AI (Business Today, 12 May 2026).
- RBI, beneficiary account name look-up for RTGS and NEFT (Statement on Developmental and Regulatory Policies, 9 October 2024; banks to comply by 1 April 2025).
- Mastercard, "How Verifiable Intent builds trust in agentic AI commerce" (2026); Pine Labs P3P launch coverage (The Paypers, June 2026).
- Concurrent work: Mool, github.com/Dipensinghh/mool (5 Oct 2026); Rudratosh, "Testing provenance-based controls against indirect prompt injection in AI agents", HackerNoon (11 Oct 2026); fps-ai-security-poc, github.com/Sarjeant/fps-ai-security-poc (19 Oct 2026).
- E. Debenedetti et al., AgentDojo (NeurIPS 2024) and Defeating Prompt Injections by Design (CaMeL), arXiv:2503.18813.
- L. Fan et al., The Granularity Mismatch in Agent Security: Argument-Level Provenance (PACT), arXiv:2605.11039; S. Weng et al., ARGUS, arXiv:2605.03378.
- Y. Louck, A. Dvir, A. Stulman, Signing the Transaction but Not the Decision (A-VIP), arXiv:2609.11757.
- A. Aviv et al., Beyond the Mandate: A Systematic Security Analysis of AP2, arXiv:2608.23858.
- aiAuthZ: Off-Host, Identity-Bound Authorization for AI Agents, arXiv:2607.05518.
- L. Beurer-Kellner et al., Design Patterns for Securing LLM Agents against Prompt Injections, arXiv:2506.08837.
- T. South et al., Authenticated Delegation and Authorized AI Agents, arXiv:2501.09674.
- Y. Nian et al., Auditable Agents, arXiv:2604.05485.